Where the Bits Go: Sensitivity-Allocated Mixed Precision for a 4B Reasoning Model on a 12 GB Consumer GPU

Eason Sun
GEHIRNS

October 2026
DOI: 10.5281/zenodo.23630199

Abstract

On consumer GPUs, reasoning language models are constrained chiefly by memory capacity and bandwidth rather than by compute. For Qwen3.5-4B, a hybrid of Gated DeltaNet and full-attention layers, we measure how sensitive each class of tensors is to quantization and spend a fixed size budget where the measured damage per byte is largest, choosing only among standard GGUF formats, so that the result needs no training and runs unmodified in Ollama. Within the range of weight sizes we tested (about 1.4–2.0 GiB), fewer bytes placed by this allocation beat more bytes placed at random: at each of three budgets the allocation diverges less from BF16 than all three random placements of the same bytes, and the allocation at 1.80 GiB diverges less than every random placement at 2.04 GiB. Against a uniform assignment of types the evidence is weaker: at the smallest budget the two tie, the allocation using 0.04 GiB less, and at the middle budget the allocation is about 1 point of divergence lower by interpolation. The resulting model, `magi-1.0`, served with an 8-bit KV cache, reduces resident GPU memory 3.4-fold relative to BF16 (3.0-fold with the default 16-bit cache), retains 96% of BF16's index on a 12-suite benchmark while its error rate rises from 16.0% to 19.4%, decodes 2.7 times as fast and achieves 3.0 times the capability per gibibyte. Relative to a published 2-bit file of similar size, its advantage comes almost entirely from fewer degenerate reasoning loops. A standard 4-bit file remains more accurate than this configuration by 3.5 index points but requires 1.13 GiB more memory, so `magi-1.0` is preferable only when memory is the binding constraint.

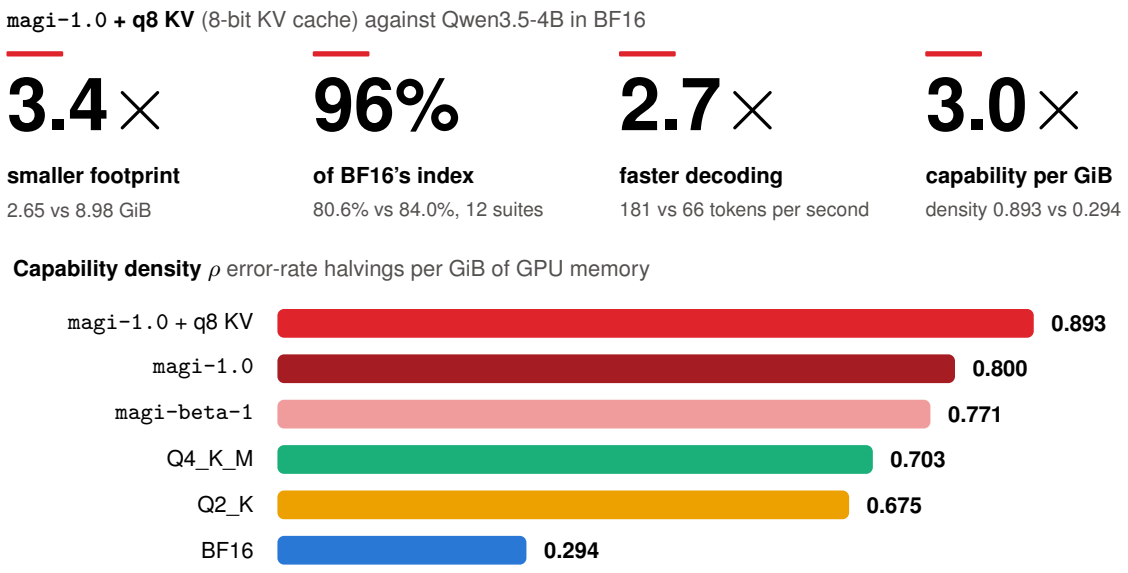


Figure 1: Served with an 8-bit KV cache (q8 KV), `magi-1.0` retains 96% of BF16's index in less than a third of its memory (0.295); with the default 16-bit cache it needs 0.337 of BF16's memory. Top: `magi-1.0` + q8 KV against the BF16 model on one RTX 5070, on the same benchmark with thinking enabled. Bottom: capability density of each configuration (95% intervals in Table 7). Q4_K_M is more accurate than `magi-1.0` but larger.

1 Introduction

A 12 GB consumer GPU can hold Qwen3.5-4B in BF16, but loaded with a 32K-token context the model occupies 8.98 GiB, which leaves little room for longer contexts, a second model, or the desktop that shares the card. Post-training quantization is the standard remedy [10, 18, 32]. In the GGUF ecosystem used by local runtimes, it is applied through presets named after a nominal bit-width, such as Q4_K_M or Q2_K, which llama.cpp defines as near-uniform type assignments with a few hand-chosen exceptions [11]; some publishers refine these assignments per tensor [33, 1].

The central claim of this paper is that, for a small hybrid reasoning model, the placement of bits across tensors matters more than their total number. In testable form: within the range of weight sizes we examined (about 1.4–2.0 GiB), a mixture whose bytes are placed according to measured sensitivity diverges less from the full-precision model than a uniform assignment of types of similar size, and less than random placements even of more bytes. If this holds, a model can be made both smaller and more accurate than a standard preset of similar size by redistributing bits rather than adding them. The claim does not extend to larger budgets: a standard 4-bit preset that needs 0.76 GiB more memory remains more accurate than our model end to end.

We test the claim on Qwen3.5-4B under one constraint: every model must run unmodified in Ollama [28], so the method can only choose among existing GGUF types for each tensor, without training, rotations or custom kernels. The resulting model, `magi-1.0`, contains Qwen's original weights stored in a different mixture of formats. We examine the claim at three levels:

- **Tokens** (Section 5.1): at three size budgets, allocated mixtures diverge less from BF16 than all three random placements of the same bytes; against uniform assignments the evidence is weaker, with a tie at the smallest budget.
- **Behaviour** (Section 5.2): on the tasks that separate a 4-bit from a 2-bit file, allocation reduces the reasoning loops behind most failures.
- **End to end** (Sections 5.3–5.5): on a 12-suite benchmark, `magi-1.0` with the default 16-bit KV cache retains 97% of BF16's index with about a third of its resident memory (96% in less than a third with an 8-bit cache).

Section 6 reports the limits of the approach, and Figure 1 summarizes the result.

2 Background

Quantization and allocation. GPTQ [10], AWQ [18] and QuIP# [32] reduce the error of quantizing each weight matrix; llama.cpp implements block-wise K-quants and lattice-based i-quants from about 1.56 to 8.5 bits per weight and weights rounding with an importance matrix [11]. Assigning precision by measured sensitivity goes back to HAWQ, which allocates bit-widths per layer [7, 8]; for language models, SliM-LLM allocates them per group of weights within a layer [15]. For GGUF files, Unsloth tunes types per tensor [33], and Addario's automatic target-bpw allocation for this model yields lower mean KL divergence than bpw-matched Q4_K_M files: the card's tables show about 12% less than Unsloth's Q4_K_M at about 5 bits per weight [1]. These studies measure divergence on reference text, not on long free-running reasoning. Rotation methods [2, 23] and KV-cache quantization [22, 14] are complementary.

Quantization and reasoning. Quantized reasoning models lose more accuracy on harder tasks but do not produce longer outputs [21], and the Qwen3 family degrades more than LLaMA-3 under low-bit quantization [39]. A visible symptom in our setting is degenerate repetition [13], which under a fixed thinking budget consumes the budget without producing an answer; we measure it throughout and discuss the apparent tension with Liu et al. in Section 7.

The model. Qwen3.5-4B interleaves three Gated DeltaNet layers [37] with one gated full-attention layer, eight times over [30]. The recurrent layers keep a fixed-size state instead of a growing cache. Recent work finds that even the gate projections of Gated DeltaNet tolerate 4-bit quantization [17]; `magi-1.0` nevertheless keeps the small α and β gate projections at Q8_0 as a cheap precaution (0.22% of its bytes).

3 Setting

Hardware and runtime. All experiments were run on a single workstation with an NVIDIA RTX 5070 (12 GB of GDDR7 on a 192-bit bus [27]; 12,227 MiB visible; 672 GB/s, inferred from the measured 28 Gbps

data rate), an AMD Ryzen 7 9800X3D and 32 GB of DDR5 under Windows 11. Models were served by Ollama 0.40.3 (installed on 11 October, before every reported run; its logs report this version at every start until writing), one at a time on the GPU. All GGUF files of our own were produced with `llama-quantize` from the PrismML-Eng fork of llama.cpp (build 10772, commit `e8fd209c9`) [29]; the formats involved are standard llama.cpp types [11].

Model. The GGUF headers give 32 blocks (24 Gated DeltaNet, 8 gated full attention), hidden size 2,560, FFN size 9,216, 16 attention heads with 4 KV heads of dimension 256, a recurrent state of $32 \times 128 \times 128$ per DeltaNet block, a 248,320-token vocabulary, and 4,205,751,296 parameters, 15.1% of them in the input embedding, which is tied to the output head. Table 3 maps the GGUF tensor names to their roles.

Size and footprint. We use three measures of size. The *file size* is that of the GGUF file on disk and is used for the allocation budgets (Tables 2 and 5, Figure 3). The *loaded weights* W are the tensor data that the runtime places on the GPU; they equal the file size up to metadata, except for bartowski's Q2_K, whose file holds 15 tensors that the runtime does not load (2.041 GiB on disk, 1.912 GiB loaded). The *footprint* F is the resident size that Ollama reports through `/api/ps` at `num_ctx` = 32,768; all densities and memory comparisons use F. It comprises weights W, KV cache K, recurrent state S and compute buffers C, $F = W + K + S + C$. For the eight attention layers,

$$K = 2\, L_{\text{attn}}\, n_{kv}\, d_h\, b\, T = 2 \cdot 8 \cdot 4 \cdot 256 \cdot 2 \cdot 32{,}768 \text{ B} = 1.000 \text{ GiB} \tag{1}$$

at 16 bits and 0.531 GiB at Q8_0. With the 50 MiB of FP32 recurrent state, this gives the 1,074 MiB that the runtime logs at load. The CUDA context of approximately 1.05 GiB, which is the same for every model, is excluded; Section 5.4 shows that including it does not change the ranking. The footprint of `magi-beta-1` was recorded only from Ollama's rounded display (3.3 GB, that is, 3.07 ± 0.05 GiB).

Benchmark and protocol. The benchmark comprises 2,313 tasks from twelve suites (Table 1). Code solutions are graded by executing tests: the EvalPlus base and extended tests for HumanEval+ and MBPP+ [4, 3, 20], and the problems' test cases for LiveCodeBench [16]. Multiple-choice suites are scored on the last answer letter, GSM8K on the final number [6], and ToolCall on the function name and the required arguments. All prompts are zero-shot. Every model thinks before answering under budget forcing [26, 36]: it may use up to 4,096 thinking tokens, after which the closing phrase *"Considering the limited time by the user, I have to give the solution based on the thinking directly now."* and the end-of-thinking tag are appended, and it then answers under a suite-specific cap (1,024 tokens for HumanEval+ and MBPP+, 2,048 for LiveCodeBench and the multiple-choice suites, 512 for GSM8K, 256 for ToolCall). Decoding is greedy, and only the answer is graded.

Table 1: Benchmark suites, used identically for every model.

Suite	Tasks	Grading	Suite	Tasks	Grading
HumanEval+ [4, 20]	164	tests	HellaSwag [38]	200	letter
MBPP+ [3, 20]	399	tests	WinoGrande [31]	200	letter
LiveCodeBench[a] [16]	150	tests	OpenBookQA [25]	200	letter
ARC-Challenge [5]	150	letter	SciQ [34]	200	letter
MMLU [12]	200	letter	TruthfulQA [19]	200	letter
GSM8K [6]	150	number	ToolCall (in-house)	100	call

[a] release_v6, the 150 newest problems; contests from 2025-01-18 to 2025-04-06.

Metrics. The *index* I is the unweighted mean of the 12 suite pass rates. We call

$$\rho = \frac{-\log_2(1 - I)}{F} \tag{2}$$

the *capability density*: the number of halvings of the error rate per resident gibibyte. The name follows Xiao et al. [35], who define the density of a model as the ratio of its effective parameter size, the size that a reference model needs to match its performance, to its actual parameter size. Our metric differs in both terms: capability is measured directly as error-rate halvings on a fixed benchmark, and it is divided by measured resident memory rather than by the number of parameters, so it credits quantization and KV-cache compression, which leave the parameter count unchanged. It is specific to this benchmark and to the 32K

context at which every footprint was measured. Configurations of equal density lie on $I(F) = 1 - 2^{-\rho F}$, and the exchange rate between accuracy and memory follows from $\partial\rho/\partial I = 1/(F(1-I)\ln 2)$ and $\partial\rho/\partial F = -\rho/F$; at `magi-1.0`'s operating point, one index point is worth 0.096 GiB. A thinking trace counts as a *loop* when its last 4,000 characters compress under zlib (default level 6) to less than 10% of their length; traces shorter than 1,000 characters never count as loops. Paired comparisons use the exact sign test on discordant pairs [24]; index differences use a paired bootstrap stratified by suite, with 4,000 resamples [9].

4 Method

4.1 A divergence instrument

Because a full benchmark run takes 8.6–23.2 hours, allocation decisions require a much faster proxy for quantization damage. We use the fraction of positions at which the quantized model's greedy next token differs from BF16's under teacher forcing (`llama-perplexity` with KL-divergence output), on three sets that share no items with the benchmark. *E-long* consists of 8 thinking traces that Qwen3.5-4B (Unsloth's Q8_0, thinking enabled) wrote for LiveCodeBench problems dated before the benchmark's contest window; the traces, each with its problem statement, are concatenated (121 KB) and cut into four 8,192-token windows whose second halves are scored (16,384 tokens), so scored tokens sit at positions 4,096–8,192 of a window, not of a trace. *E-chat* consists of the same model's answers to 30 everyday prompts (8,192 scored tokens), and *E-code* of 46 LiveCodeBench problem statements older than the benchmark's contests (9,216 scored tokens). The importance matrix of Section 4.3 uses 32 further traces of the same kind, disjoint from E-long. The objective J is the mean of the E-long and E-chat rates. E-code is not part of J; it serves as a held-out check and in the single-layer probe of Section 6. The instrument orders the published files by size on all three sets, with Q8_0 near zero (Table 2), and it agrees with the benchmark for the two published files that we benchmarked, Q4_K_M and Q2_K. Perplexity is unsuitable as a proxy: Q3_K_M obtains a perplexity ratio of 0.926 against BF16 on E-code.

Table 2: Top-1 disagreement with BF16 (%) for published quantizations of Qwen3.5-4B. E-code is not part of the objective J.

Quantization	File (GiB)	E-long	E-chat	E-code
Q8_0 (Unsloth)	4.175	0.66	1.10	1.59
Q4_K_M (Unsloth)	2.553	3.10	5.33	5.68
Q3_K_M (Unsloth)	2.136	5.56	9.92	8.39
Q2_K (bartowski)	2.041	9.98	18.42	13.09
UD-IQ2_XXS (Unsloth)	1.416	18.02	27.83	18.77

4.2 Sensitivity map

Every tensor is quantized once to each candidate type with Unsloth's importance matrix, and mixtures are spliced from this library; spliced files are byte-identical to `llama-quantize` output with the same types (426 of 426 tensors). We start from two reference mixtures with the DeltaNet gates at Q8_0: a floor with IQ2_XXS bodies and a Q4_K embedding (1.196 GiB of tensor data, $J = 27.80$) and a ceiling with Q4_K bodies and a Q6_K embedding (2.361 GiB, $J = 4.92$). From these we raise one tensor class from the floor or lower one from the ceiling. The 44 resulting mixtures give the change in J per gibibyte for each class. The two directions agree on the four most and the two least sensitive classes, but the middle of the ranking reorders (Spearman rank correlation 0.84 over the 11 classes).

The values differ widely (Figure 2): lowering the attention value projection from Q4_K to IQ2_XXS costs 437 points of J per gibibyte saved and lowering the FFN gate costs 22, a ratio of about 20; lowering the tied embedding from Q6_K to Q4_K, a milder step, costs only 2.0. Protecting the small attention projections and the DeltaNet output projection therefore requires few bytes, and the FFN gate is the least sensitive of the large classes. Sensitivity also varies with depth: raising one quarter of the layers at a time helps most in layers 8–15, for both the FFN and the DeltaNet input projections. Because the effects are not additive, every candidate mixture is measured directly.

4.3 Allocation and the resulting model

The allocator starts at the floor and repeatedly applies, among the upgrades that still fit in the budget, the one with the largest measured decrease in J per gibibyte, until none fits. An upgrade moves either a

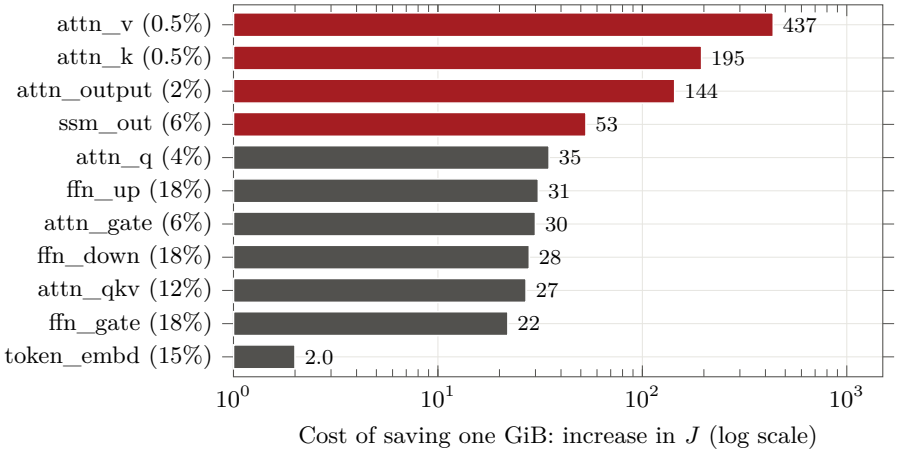


Figure 2: The most sensitive tensor classes are among the smallest. Increase in the divergence objective J per gibibyte saved when one tensor class is lowered from the Q4_K ceiling to IQ2_XXS (the embedding: from Q6_K to Q4_K, a milder step). Shares of parameters are in brackets, and Table 3 gives the role of each GGUF tensor name. The four most sensitive classes (red) hold 9% of the parameters, so protecting them requires few bytes.

whole tensor class to a higher type (Q2_K, IQ3_XXS or Q4_K; for the embedding, Q4_K to Q6_K) or the FFN or DeltaNet input projections of one quarter of the layers to Q4_K. The value of each upgrade is taken from the single-change measurements of Section 4.2 and is not re-measured after each step, so the allocator assumes additivity; because its predictions were too optimistic at every budget, each mixture is judged by its measured J. The random controls follow the same procedure but choose the next upgrade uniformly at random among those that fit (seeds 1–3), so they spend the same budget on the same menu of upgrades. We ran the allocator at three file-size budgets: S = 1.416 GiB, the size of Unsloth's UD-IQ2_XXS file; L = 2.041 GiB, the size of bartowski's Q2_K file; and M = 1.80 GiB, an intermediate budget that does not correspond to a published file. For M we also computed the importance matrix from 32 of the model's own reasoning traces, which lowered E-long disagreement from 7.52% to 7.01% compared with Unsloth's matrix but left E-chat essentially unchanged (12.93% and 12.99%).

`magi-1.0` is this M mixture (Table 3). The FFN projections and the DeltaNet output gate are stored in IQ3_XXS (3.06 bits per weight); the full-attention projections, the DeltaNet input and output projections and the tied embedding in Q4_K (4.5); the DeltaNet gates α and β in Q8_0; and norms, convolutions and the A and Δt parameters in FP32. The all-in rate is 3.650 bits per weight, or 1.787 GiB of tensor data. A second model, `magi-beta-1`, differs only in storing the full-attention key, value and output projections at Q6_K (1.817 GiB, 3.712 bits per weight). Table 4 defines the names used for the other configurations.

Table 3: Composition of `magi-1.0` by GGUF tensor name, read from the released file. Columns may not sum exactly because of rounding.

GGUF tensor	Role	Layers	Type	Params (%)	Bytes (%)	MiB
`token_embd`	embedding, tied to the output head	—	Q4_K	15.11	18.64	341.0
`ffn_gate/up/down`	FFN projections	32 each	IQ3_XXS	53.85	45.19	826.9
`attn_qkv`	DeltaNet input projection (q, k, v)	24	Q4_K	11.97	14.75	270.0
`attn_gate`	DeltaNet output gate	24	IQ3_XXS	5.98	5.02	91.9
`ssm_out`	DeltaNet output projection	24	Q4_K	5.98	7.38	135.0
`ssm_alpha/beta`	DeltaNet gates α, β	24 each	Q8_0	0.09	0.22	4.0
`attn_q`	attention query + output gate (fused)	8	Q4_K	3.99	4.92	90.0
`attn_k/v`	attention key, value	8 each	Q4_K	1.00	1.23	22.5
`attn_output`	attention output projection	8	Q4_K	1.99	2.46	45.0
FP32 (177)	norms, convolution, A, Δt bias	—	FP32	0.02	0.20	3.7
Total (426)	3.65 bits per weight			100.00	100.00	1,829.9

Table 4: Names of the configurations compared in this paper.

Name	Definition
S, M, L	File-size budgets of 1.416, 1.80 and 2.041 GiB for the allocator (Section 4.3).
greedy S, M, L	Allocator output at that budget, with Unsloth's importance matrix.
random S, M, L	The same procedure with random upgrade orders (seeds 1–3). In Section 5.2, *random L* is the L mixture with the lowest J (seed 3, $J = 12.96$).
uniform body	Every tensor except the embedding and the DeltaNet gates at one type (S: Q2_K, M: IQ3_XXS, L: Q3_K), including the sensitive attention and DeltaNet output projections; α and β at Q8_0 and the embedding at Q4_K, as in the floor (L: Q6_K, as in the ceiling).
`magi-1.0`	Greedy M with the importance matrix from the model's own traces.
`magi-beta-1`	`magi-1.0` with the full-attention key, value and output projections at Q6_K; its pre-release build used Unsloth's importance matrix.
+ q8 KV	The same weights served with an 8-bit KV cache (flash attention enabled).
candidate d	`magi-1.0` with the FFN of layers 0, 2 and 27–29 at IQ1_M, the FFN of layers 24–26 and 30 at IQ2_XXS, the other FFN gates outside layers 8–15 at IQ2_XXS, and the full-attention key, value and output projections at Q6_K (1.699 GiB file).
decisive, Q2-only, controls	The three task groups of the behavioural screen (Section 5.2).

5 Results

5.1 Allocation beats random placement of the same bytes

At each of the three budgets, the greedy allocation diverges less from BF16 than all three random allocations of the same bytes (±0.02 GiB), on both evaluation sets (Table 5, Figure 3). If the greedy mixture were merely a fourth random draw, the probability that it would beat all three random draws at each of the three budgets would be $(1/4)^3 \approx 0.016$. The comparison also holds across budgets: the greedy M mixture (1.797 GiB, $J = 10.00$) diverges less than every random L mixture (2.03–2.04 GiB, $J = 12.96$–13.30). Within this range, fewer bytes placed well therefore beat more bytes placed at random. (Both allocators stop when no upgrade fits. Because the leftover budget depends on the order of upgrades, the random mixtures end 0.011–0.014 GiB larger at S and 0.003–0.011 GiB larger at L, which favours them.)

Random placement is nevertheless a weak baseline: even the uniform body at 1.622 GiB ($J = 12.18$), which spends nothing extra on the sensitive tensors, diverges less than every random M mixture. Against the uniform body, which shares the floor's settings and importance matrix, greedy ties at S, with 0.04 GiB fewer bytes: its J is 0.37 points lower, about one binomial standard error of the difference (0.39), and it is clearly lower on E-long (14.01 ± 0.27 against 15.35 ± 0.28) but slightly higher on E-chat (24.83 ± 0.48 against 24.23 ± 0.47). At M and L the uniform mixtures are smaller (1.622 and 1.930 GiB); linear interpolation between them puts a uniform mixture of 1.797 GiB at $J \approx 11.2$, about 1 point above greedy M with the same matrix (10.23), an estimate only, because the L mixture also changes the format (Q3_K) and the embedding (Q6_K). At L, greedy is 2.9 points lower with 0.10 GiB more; extrapolating the same line to greedy L's size gives a margin of about 2.4 points. Against published files, the greedy L mixture halves the divergence of bartowski's Q2_K file of the same file size (which loads 0.11 GiB less, Section 3) and matches Unsloth's larger Q3_K_M; at budget M, `magi-1.0` is smaller than the Q2_K file and closer to BF16. The own-trace importance matrix contributes part of `magi-1.0`'s advantage, but with Unsloth's matrix, which the random M mixtures also used, the greedy M mixture still reaches $J = 10.23$, below every random M mixture.

5.2 Allocation reduces the loops that quantization causes

Divergence measured under teacher forcing cannot capture errors that compound over thousands of free-running tokens. We therefore built a behavioural screen from the full benchmark runs of Unsloth's Q4_K_M and bartowski's Q2_K. It contains 86 *decisive* tasks that Q4_K_M solved and Q2_K failed (31 HumanEval+, 30 LiveCodeBench, 25 MMLU), 10 *Q2-only* tasks with the opposite outcome, and 30 *controls* that both solved, drawn at random (10 per suite). The decisive and Q2-only tasks are all discordant tasks of these three suites. MBPP+, the suite with the largest difference, was not included because the Q4_K_M run had not yet reached it when the screen was built. Because the decisive tasks were selected for Q2_K failing them, any other model will recover some of them by chance; a random allocation of the same bytes as greedy L measures how many. The 126 screen tasks are also part of the full benchmark; Section 5.3 reports the benchmark with and without them.

Q2_K loops on 49 of the 86 decisive tasks, whereas Q8_0 and Q4_K_M loop on one and three (Table 6,

Table 5: Divergence objective J (mean of E-long and E-chat top-1 disagreement, %) at three file-size budgets.

Budget	Mixture	File (GiB)	E-long	E-chat	J
S	published UD-IQ2_XXS	1.416	18.02	27.83	22.93
	greedy	1.401	14.01	24.83	**19.42**
	random allocations (3)	1.412–1.415	15.48–18.99	26.59–30.77	21.04–24.88
	uniform Q2_K body	1.440	15.35	24.23	19.79
M	greedy, own-trace imatrix (`magi-1.0`)	1.797	7.01	12.99	**10.00**
	greedy, Unsloth imatrix	1.797	7.52	12.93	10.23
	random allocations (3)	1.794–1.798	10.52–13.38	18.15–21.98	14.33–17.54
	uniform IQ3_XXS body	1.622	9.14	15.23	12.18
L	published Q2_K (bartowski)	2.041	9.98	18.42	14.20
	published Q3_K_M (Unsloth)	2.136	5.56	9.92	7.74
	greedy	2.029	5.38	9.75	**7.56**
	random allocations (3)	2.032–2.040	9.34–9.98	16.04–16.77	12.96–13.30
	uniform Q3_K body, Q6_K embedding	1.930	7.36	13.59	10.47

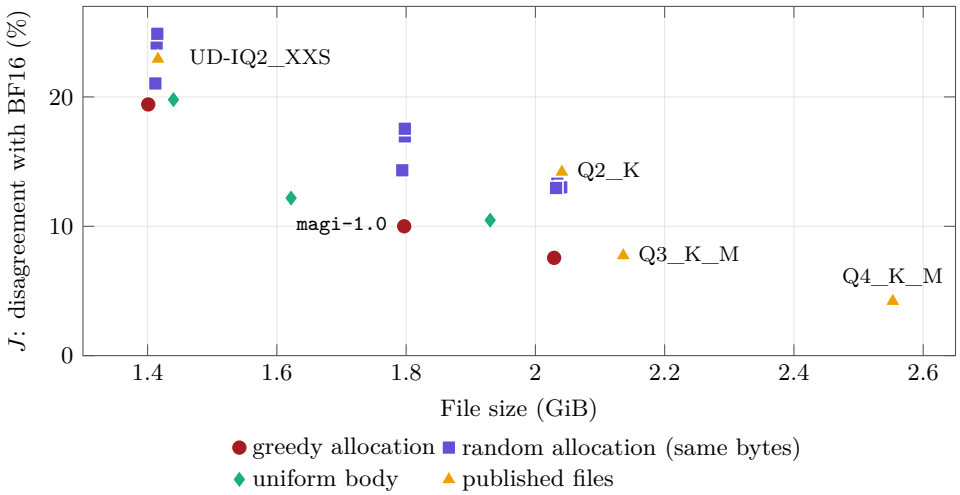


Figure 3: Divergence against file size. At each budget, the greedy allocation (red) diverges less from BF16 than all three random allocations of the same bytes (violet) and than the published files of similar size (amber); the larger published Q4_K_M file ($J = 4.21$) diverges less than any greedy mixture. The red point at M is `magi-1.0`, built with the own-trace importance matrix ($J = 10.00$); the random M mixtures used Unsloth's matrix, with which greedy M reaches $J = 10.23$. Lower is better.

Figure 4). Q4_K_M's low count partly reflects the selection, but Q8_0, which played no part in it, shows that greedy decoding and the thinking budget alone do not produce the loops. Allocation removes most of these loops: a random allocation at budget L solves 36 decisive tasks and loops on 28; the greedy allocation of the same size and importance matrix solves 65 and loops on 4 (31 against 2 discordant solves and 1 against 25 discordant loops, $p < 10^{-6}$ for both). `magi-1.0`, which is smaller than both and uses the own-trace matrix, solves 58 and loops on 9. Paired against the random control, it solves 26 tasks that the control misses against 4 the other way ($p < 0.0001$), loops on 4 tasks on which the control does not against 23 the other way ($p = 0.0003$), and still solves all 30 controls. Allocation reduces loops but does not eliminate them: `magi-1.0` still loops on 9 of the 86 decisive tasks, and 99 of its 463 failures on the full benchmark end in a loop.

Figure 5 illustrates this failure mode on a single MBPP+ task. Q2_K settles on what to write and then repeats the same three lines until its 4,096-token budget is exhausted, and the answer it is then forced to give uses the wrong function name. `magi-1.0` reaches the one-line solution after 525 tokens of thinking.

5.3 Full benchmark

Table 7 and Figure 6 report all 12 suites. `magi-1.0` scores 81.3%, 2.7 points below BF16 and 4.2 points above Q2_K, although it loads fewer weight bytes than Q2_K (1.787 against 1.912 GiB). Both differences are significant (paired 95% intervals 1.45–3.86 and 2.83–5.60). The largest single loss against BF16 is on

Table 6: Behavioural screen: 86 decisive tasks, 10 Q2-only tasks and 30 controls (thinking budget 4,096, greedy decoding). Footprints at 32K context.

Model	Footprint (GiB)	Decisive: solved	loops	Q2-only	Controls
Q2_K, bartowski (published)	3.148	0	49	10	30
Random L	3.258	36	28	8	26
`magi-1.0` (greedy M)	**3.024**	**58**	**9**	8	**30**
Greedy L	3.255	65	4	5	30
Q8_0, Unsloth	5.62	77	1	3	29
Q4_K_M, Unsloth (defines the set)	3.779	86	3	0	30

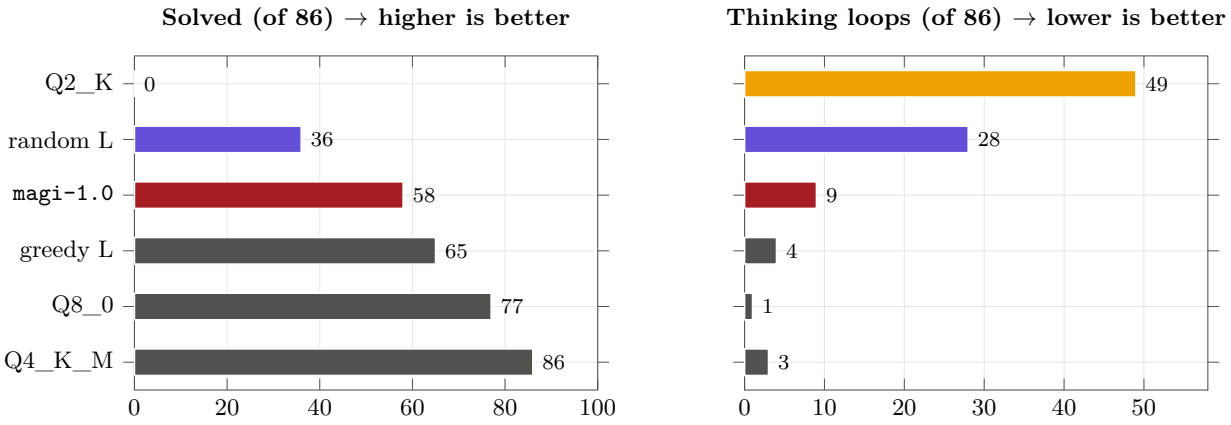


Figure 4: Allocation reduces the loops that break quantized reasoning. On the 86 tasks that Q2_K fails and Q4_K_M solves, Q2_K loops on 49 and the random allocation on 28; `magi-1.0`, which is smaller than both, loops on 9 and solves 58. Random L has the same bytes as greedy L.

Task (MBPP+/105): *Write a python function to count true booleans in the given list.* `assert count([True,False,True]) == 2`

Q2_K thinking: **4,096 tokens (budget exhausted)**

. . . final lines of the trace:

```
Okay, I will write the function. I will also write the test
case to ensure it runs correctly.
Actually, I'll just write the function. The assertion is
part of the problem description.
Wait, I'll write the function. The assertion is part of the
problem description.
Okay, I will write the function. I will also write the test
case to ensure it runs correctly.
Actually, I'll just write the function. The assertion is
part of the problem description.
Wait, I'll write the function. The assertion is part of the
problem description.
```

(the three lines repeat until the budget ends)

Answer: `def count_true(lst): ...`

FAIL: the tests call `count`, which the answer never defines.

`magi-1.0` thinking: **525 tokens**

```
The user wants a Python function that counts the number of
True booleans in a given list. The function should be named
count based on the assertion provided.
...I can use sum() with a generator expression or
list.count() or a loop.
...True is 1, False is 0. sum([True, False, True]) -> 1 + 0
+ 1 -> 2. Correct.
```

Answer:

```
def count(lst):
   return sum(lst)
```

PASS.

BF16 also passes, after 1,826 tokens of thinking.

Figure 5: A degenerate loop. On the same task, Q2_K repeats itself until its thinking budget is exhausted, whereas `magi-1.0` answers correctly after 525 tokens (verbatim excerpts). Over the full benchmark, 59% of Q2_K's failed tasks end in a loop of this kind (337 of 575), against 21% for `magi-1.0` (99 of 463).

LiveCodeBench, where `magi-1.0` solves 37 of 150 problems against 53; this suite accounts for about a third of the summed per-suite gap (10.7 of 31.9 points), and on the other eleven suites `magi-1.0` trails by 1.9 points on average. Neither of the two variants changes the result materially. The 8-bit KV cache lowers the index by 0.66 points (61 tasks solved only with it and 77 only without it; sign test $p = 0.20$) and saves 0.37 GiB; `magi-beta-1`, which spends slightly more on the attention projections, scores 0.65 points lower than `magi-1.0` (83 against 92 discordant tasks, $p = 0.55$). Neither difference is significant at this sample size.

Task by task, `magi-1.0` solves 209 tasks that Q2_K fails and fails 97 that Q2_K solves, with the largest gains on MBPP+, HumanEval+ and MMLU. Against BF16 the balance is 73 to 138, with the largest deficit again on LiveCodeBench.

The 126 screen tasks are part of this benchmark, and they were used both to admit `magi-1.0` to the full run and to set the screening thresholds of Section 6. Without them (2,187 tasks), `magi-1.0` scores 80.30% and trails BF16 (82.64%) by 2.34 points (67 against 121 discordant tasks, $p = 0.0001$), and it leads Q2_K (78.90%) by 1.40 points (151 against 95, $p = 0.0004$). This exclusion removes 86 tasks that Q2_K fails and Q4_K_M solves by construction, so it is biased towards Q2_K and against Q4_K_M. `magi-1.0` with the 8-bit cache remains the configuration with the highest density (0.878).

The standard 4-bit preset is a strong baseline. Unsloth's Q4_K_M (84.14%) differs from BF16 by only 0.18 points (75 against 71 discordant tasks, $p = 0.80$), and it is significantly more accurate than `magi-1.0`, by 2.85 points (132 against 63, $p < 10^{-5}$). It also needs 0.76 GiB more memory than `magi-1.0`, or 1.13 GiB more than `magi-1.0` with the 8-bit cache. When accuracy matters more than memory, it is the better choice. When memory is the constraint, `magi-1.0` gives up less accuracy than its memory saving is worth, which is what the density metric measures.

Table 7: Pass rates (%) on the full benchmark (2,313 tasks). Index intervals are 95% stratified bootstrap intervals; density intervals are propagated from them.

Suite	BF16	Q4_K_M	Q2_K	`magi-1.0`	`magi-1.0` + q8 KV	`magi-beta-1`
HumanEval+	91.46	92.68	75.00	87.80	88.41	87.20
MBPP+	72.18	73.68	58.90	68.42	67.92	70.43
LiveCodeBench	35.33	35.33	15.33	24.67	24.00	20.00
ARC-Challenge	98.67	98.67	97.33	97.33	98.00	98.00
MMLU	84.50	82.50	74.00	82.00	81.00	81.50
HellaSwag	75.50	72.00	71.50	72.00	69.00	68.00
WinoGrande	87.00	84.50	80.00	82.50	84.00	83.00
OpenBookQA	95.00	96.50	93.00	96.00	95.00	95.50
SciQ	98.00	98.50	96.00	96.00	97.00	98.00
TruthfulQA	82.50	82.00	73.00	79.50	76.00	77.50
GSM8K	87.33	93.33	90.67	89.33	87.33	88.67
ToolCall	100.00	100.00	100.00	100.00	100.00	100.00
Index	83.96	**84.14**	77.06	81.30	80.64	80.65
95% interval	82.63–85.26	82.86–85.40	75.56–78.44	79.99–82.62	79.29–82.05	79.31–81.95
Weights (GiB)	7.836	2.553[a]	1.912	1.787	1.787	1.817
Footprint at 32K (GiB)	8.978	3.779	3.148	3.024	2.653	3.07[b]
Capability density ρ	0.294	0.703	0.675	0.800	**0.893**	0.771[b]
95% interval	0.281–0.308	0.673–0.734	0.646–0.703	0.768–0.835	0.856–0.934	0.740–0.804

[a] File size; the other weight sizes are the tensor data that the runtime loads.

[b] From Ollama's rounded display (±0.05 GiB); the tensor data suggest about 3.05 GiB (density 0.776).

5.4 Capability per gibibyte

Figure 7 places each configuration against curves of constant density, and Table 7 gives the densities with 95% intervals. BF16 reaches 0.294, and the two standard presets reach about 0.7 by different routes: Q4_K_M keeps BF16's accuracy in 42% of its memory (0.703), whereas Q2_K saves more memory but loses 6.9 points (0.675). `magi-1.0` reaches 0.800 by being both smaller than Q2_K and more accurate, and the 8-bit cache raises it to 0.893, three times BF16's density; this interval (0.856–0.934) does not overlap those of either preset. Part of this ratio comes from the KV cache rather than from the weight allocation: with the default 16-bit cache, `magi-1.0` has a 2.97-fold smaller footprint than BF16 and 2.72 times its density. In terms of errors, `magi-1.0` with the 8-bit cache retains 96% of BF16's index, but its error rate rises from 16.0% to 19.4%, that is, by 21% (by 17%, to 18.7%, with the 16-bit cache).

The ratio depends on the accounting, whereas the ranking hardly does. Counting the CUDA context of approximately 1.05 GiB in every footprint lowers the density ratio between `magi-1.0` with the 8-bit cache and BF16 from 3.04 to 2.43; dropping ToolCall, on which every model is at its ceiling, gives 3.02; and excluding the 126 screen tasks gives 3.12. In all three cases the ranking of the six configurations is the same as in Table 7, except that without the screen tasks Q2_K (0.713) moves ahead of Q4_K_M (0.657), for the reason given in Section 5.3. All densities refer to the 32K context at which the footprints were measured.

5.5 Failures, speed and free memory

Failures. Most of `magi-1.0`'s failures are wrong answers rather than runaway reasoning (Figure 8a). It fails 463 tasks against Q2_K's 575, and 99 of these end in a loop, against 337 for Q2_K. Relative to Q2_K,

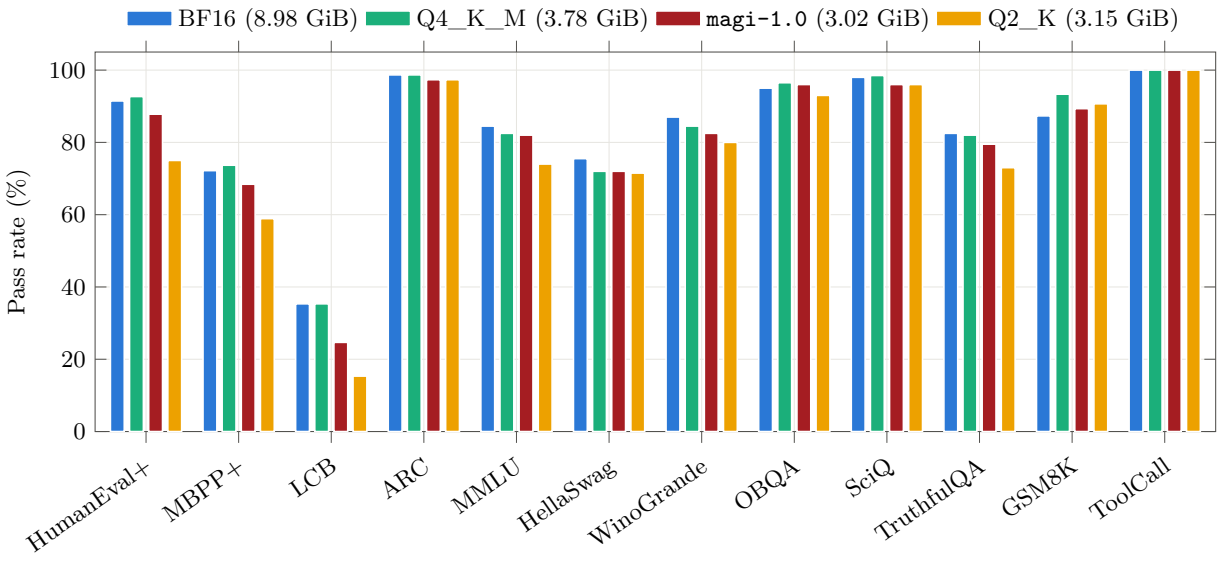


Figure 6: `magi-1.0` stays close to BF16 on most suites; its largest loss is on LiveCodeBench, whereas Q2_K falls behind on all three coding suites. Pass rate per suite; all values, including those for `magi-1.0` with the 8-bit cache and `magi-beta-1`, are in Table 7.

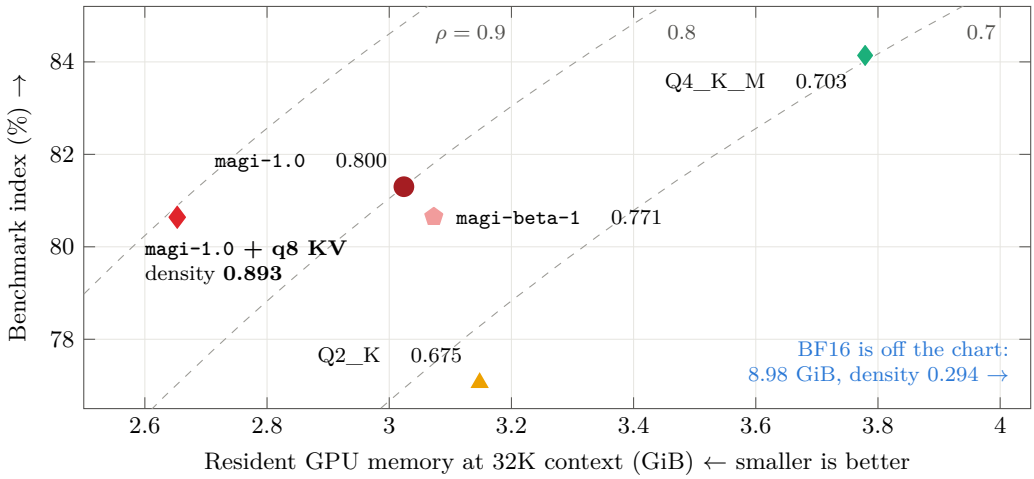


Figure 7: `magi-1.0` with the 8-bit cache has the highest capability per GiB. Index against resident GPU memory; dashed curves have constant density, $I = 1 - 2^{-\rho F}$. Up and to the left is better. The footprint of `magi-beta-1` is approximate (±0.05 GiB).

its advantage therefore lies almost entirely in fewer loops: it has 238 fewer looping failures but 126 more non-looping wrong answers, a net difference of 112 tasks. This is an accounting identity, not a causal analysis. BF16 and Q4_K_M rarely loop (13 and 18 failed tasks).

Speed. `magi-1.0` decodes 180 tokens per second, against 147 for Q4_K_M and 66 for BF16 (Figure 8b). Summed over all tasks, the full benchmark takes 8.9 hours of wall time, including prompt processing, against 10.6 and 23.2 hours (8.7, 10.4 and 23.0 hours of decoding alone). Q2_K decodes more slowly than `magi-1.0` despite its lower nominal bit-width. It loads more weight bytes (1.912 against 1.787 GiB), and its loops may contribute, since they keep traces at the full budget (Q2_K exhausts the thinking budget on 47.5% of tasks, `magi-1.0` on 34.9%) and longer contexts make each decoding step more expensive; because throughput was recorded during benchmark runs rather than in controlled tests, we do not separate these causes. BF16 decoding is close to the bandwidth bound: it streams its weights at 83% of the card's 672 GB/s, whereas the quantized models reach 50–60%, as dequantization and per-token attention and state updates take a larger share of the time.

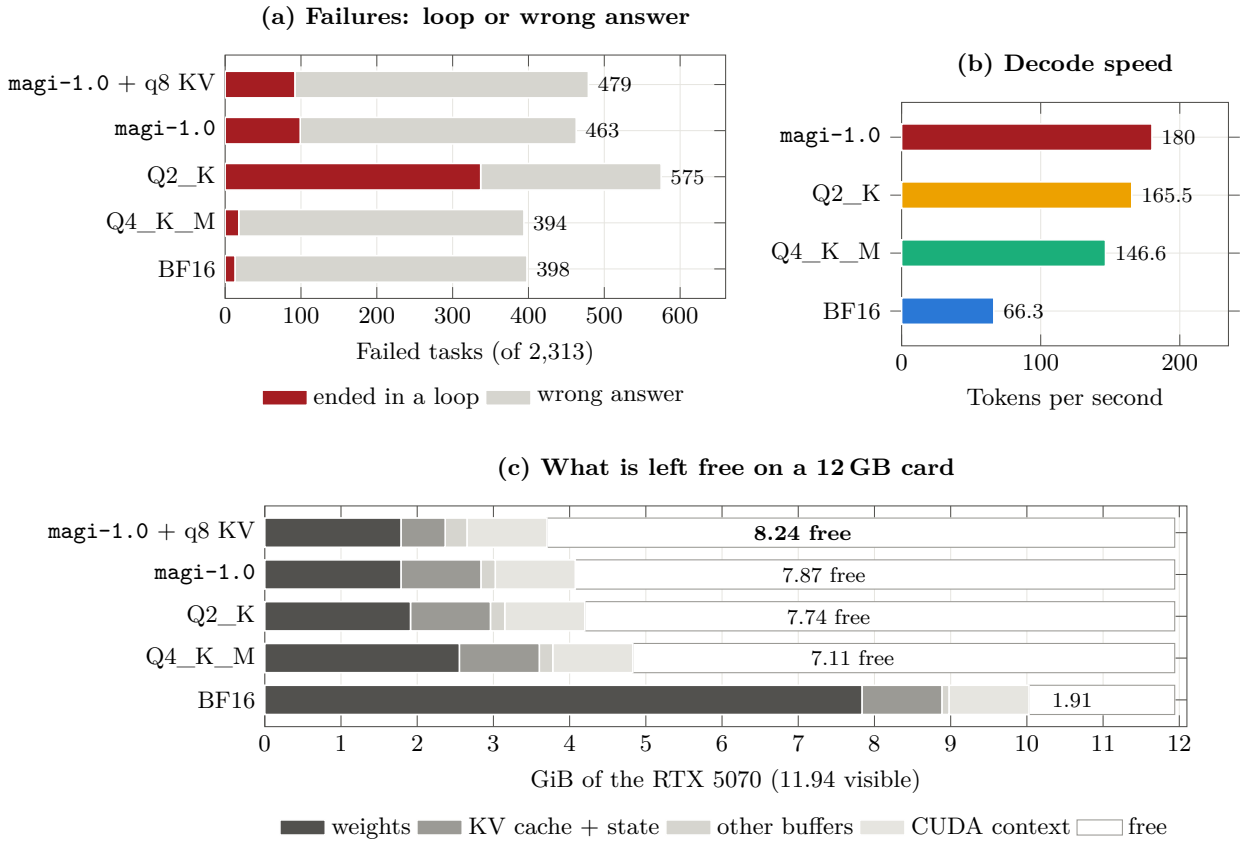


Figure 8: Failures, decode speed and free memory. (a) Failed tasks, divided into those whose thinking ends in a loop and other wrong answers. (b) Aggregate decode throughput: total generated tokens divided by total decoding time over each full run. (c) Resident footprint divided into its parts, plus the CUDA context and the free remainder.

Free memory. Counting the CUDA context, `magi-1.0` with the 8-bit cache leaves 8.2 GiB of the card free, against 7.1 GiB for Q4_K_M and 1.9 GiB for BF16 (Figure 8c). The 8-bit cache saves 0.371 GiB, less than the 0.469 GiB predicted by the cache size, because the runtime's other buffers grow from 0.188 to 0.286 GiB when it switches to the quantized cache with flash attention.

6 Limits of further compression

The sensitivity map suggests further savings. Table 8 lists the variants screened against `magi-1.0`, and Table 9 (Appendix A) every candidate of the search for a smaller global allocation. Before a full run, each candidate had to pass a gate: at least 52 decisive tasks solved, at most 13 loops and at least 28 controls, and, in the global search, at most 2 of 48 chat replies flagged by automatic checks. These thresholds were set after `magi-1.0`'s own screen (58, 9 and 30) as a tolerance around it.

- **Further reducing the least-sensitive class.** Lowering the FFN gates of layers 24–31 to IQ2_XXS saves 22.5 MiB. Solves do not change significantly, but loops rise from 9 to 20 ($p = 0.019$); because this build also used Unsloth's matrix, the rise cannot be attributed to the FFN gates alone. To raise density, a saving this small could afford to lose at most 0.23 index points. An IQ1_S variant was stopped after 86 of the 126 tasks, with no reason recorded, and is not reported.
- **Aggressive quantization of single layers.** When the FFN of one layer at a time is lowered from IQ3_XXS to IQ1_M, only layers 28 and 29 stay below the pre-specified threshold of 0.30 points of additional E-code disagreement, and each would save only 0.011 GiB; the final layer is the most sensitive (+1.64 points, against at most +1.06 for any other layer). Removing layers was not possible, because the available toolchain cannot prune this hybrid architecture without breaking it.
- **A tighter global allocation.** Of nine candidates (Table 9), five exceeded a pre-specified divergence guard, and of the others only d was screened (Appendix A). Candidate d, 0.098 GiB smaller than

`magi-1.0`, loses significantly on both solves and loops ($p = 0.011$ each); two follow-up builds, described below, trace its loops to its FFN reductions.

- **A longer thinking budget** of 8,192 tokens, tested on `magi-1.0` with the 8-bit cache, adds 26 loops and removes none ($p = 3 \times 10^{-8}$) and does not significantly improve solves (59 against 54, $p = 0.18$). Without a BF16 or Q4_K_M control at this budget, this test cannot separate an effect of compression from an effect of the budget.

Candidate d makes two changes to `magi-1.0`: it reduces the FFN and stores the full-attention key, value and output projections at Q6_K. Two follow-up builds separate them. The first keeps d's attention projections but restores `magi-1.0`'s FFN; it is the pre-release build of `magi-beta-1` discussed below, and it passed the gate. Because it used Unsloth's matrix, the comparison below uses in its place the released `magi-beta-1`, which has the same types and the own-trace matrix (outcomes from its full run). The second keeps d's FFN reductions but stores the attention projections at `magi-1.0`'s Q4_K. A grader failure left its HumanEval+ answers ungraded; on the other 55 decisive tasks it solves 22 against `magi-1.0`'s 32 (3 against 13 discordant solves, $p = 0.021$). With the own-trace matrix throughout, the four builds form a two-by-two comparison of loops on the 86 decisive tasks. The FFN reductions raise the loops at both attention precisions: from 9 to 27 with Q4_K (21 against 3 discordant loops, $p = 0.0003$) and from 10 to 22 with Q6_K (candidate d against the released `magi-beta-1`, 16 against 4, $p = 0.012$). Raising the attention projections to Q6_K changes neither count significantly: 10 against 9 with `magi-1.0`'s FFN (6 against 5, $p = 1.0$) and 22 against 27 with d's reductions (10 against 15, $p = 0.42$). The loops follow the FFN reductions, not the attention precision.

`magi-beta-1` was released after its pre-release build passed the gate (63, 13 and 29; 1 of 48 chat replies flagged). The released file has the same types but `magi-1.0`'s own-trace matrix and was not screened itself; on the 126 tasks, its full run falls one task short of the gate (51, 10 and 29) and does not differ significantly from `magi-1.0` (6 against 13 discordant solves, $p = 0.17$). Since repeated runs of one file gave identical outputs (Section 7), the 30 tasks that one build solves and the other fails (7 against 19 among the decisive tasks, $p = 0.029$) show that the screen responds to small changes of the weights; results within a few tasks of the gate, such as the released file's 51 solves or the pre-release build's 13 loops, lie within this variation between builds. `magi-beta-1` with the 8-bit cache was also screened; a grader failure left its HumanEval+ answers ungraded, but its 14 loops alone exceed the gate.

The 8-bit KV cache is the one change that received a full run despite failing the gate (54 solved, 16 loops). This exception was decided after the screen, for two reasons: its loop increase on the decisive tasks (10 against 3, $p = 0.092$) was not significant, so the screen and the full benchmark did not clearly disagree, and the gate measures behaviour but not memory; the run itself was executed as the pre-specified fallback of the global search after candidate d failed. By the exchange rate of Section 3, a saving of 0.371 GiB can afford a loss of about 4.3 index points at equal density (3.8 at the linear rate), whereas the FFN-gate change above could afford only 0.23; a principled gate would scale the tolerated loss with the memory saved. On the full benchmark the cache lowered the index by 0.66 points and raised density by 11.6%.

Table 8: Boundary experiments on the behavioural screen, paired against `magi-1.0` (16-bit cache, 4,096-token budget) unless noted.

		Screen (126 tasks)			Paired: gained/lost (p)	
Variant	Saved	Solved/86	Loops	Controls	solves	loops
`magi-1.0`	—	58	9	30	reference	reference
+ q8 KV cache	0.371 GiB	54	16	30	5/9 (0.42)	10/3 (0.092)
+ q8 KV, 8,192 budget[a]	0.371 GiB	59	42	30	7/2 (0.18)	26/0 (3×10^{-8})
FFN gate L24–31 to IQ2_XXS[c]	22.5 MiB	62	20	30	12/8 (0.50)	15/4 (0.019)
Candidate d	0.098 GiB	45	22	29	5/18 (0.011)	18/5 (0.011)
d with `magi-1.0`'s attention[e]	0.128 GiB	22/55	27	17/20	3/13 (0.021)	21/3 (0.0003)
pre-release `magi-beta-1`[c]	≈ -0.03 GiB[b]	63	13	29	11/6 (0.33)	7/3 (0.34)
`magi-beta-1`, released[d]	≈ -0.03 GiB[b]	51	10	29	6/13 (0.17)	6/5 (1.0)

Savings are footprint differences, except for the FFN-gate row (file size) and the row marked *e* (tensor data).

[a] Paired against q8 KV at 4,096.

[b] Difference in tensor data; the footprint of `magi-beta-1` is known only to ±0.05 GiB.

[c] Built from tensors quantized with Unsloth's importance matrix; all other rows use `magi-1.0`'s own-trace matrix.

[d] Not screened separately; outcomes on the 126 tasks are taken from its full benchmark run.

[e] Candidate d's FFN reductions with Q4_K attention; solves and controls exclude HumanEval+ (ungraded).

7 Discussion and limitations

Within the range tested, the evidence supports the claim against random placement at the level of tokens and of behaviour; against a uniform assignment of types it supports it only partly, and only at the level of tokens. Under teacher forcing, lowering a tensor class from Q4_K to IQ2_XXS costs about 20 times more divergence per gibibyte in the most sensitive class than in the least sensitive one, and allocation that follows these measurements beats all three random placements of the same bytes at each budget, and the one random placement that was screened. The comparison with uniform assignment is closer: at S the two tie on J, the allocation needing 0.04 GiB less, and at M its margin of about 1 point rests on interpolation, so the evidence that sensitivity, rather than any non-random placement, drives the result is weaker. On the full benchmark, a model with fewer weight bytes than a published 2-bit file is 4.2 points more accurate (1.4 points without the screen tasks). No random allocation was run on the full benchmark, so the comparison with random placement rests on divergence and on the screen. The comparisons with Q2_K are also not pure tests of placement, since the format family (IQ3_XXS lattices against K-quants), the importance matrix and the byte count all differ, but none gives the allocated model more bits.

Placement plausibly matters here because the most sensitive tensors are small. The full-attention value, key and output projections hold 3% of the parameters and the DeltaNet output projection 6%. The largest tensor, the tied embedding, changes the divergence little when lowered from Q6_K to Q4_K, whereas the published files of this model that we inspected store it at Q5_K or Q6_K. The behavioural screen was necessary because divergence, although it ordered Q4_K_M, `magi-1.0` and Q2_K correctly, cannot anticipate the loops that build up over long generations; moreover, only a few mixtures have both a divergence and a benchmark measurement. One of them, `magi-beta-1`, is indistinguishable from `magi-1.0` on J (9.93 against 10.00, a gap of about a quarter of the binomial standard error of the difference; measured on a build with the released file's types, importance matrix and size) yet scores 0.65 index points lower (not significant), so differences in J this small do not predict behaviour. Liu et al. report that quantized reasoning models do not produce longer outputs [21]. In our runs, the 4-bit file exhausts the thinking budget about as often as BF16 (on 31.7% and 31.6% of tasks), and only the 2-bit file does so much more often (47.5%), so the two observations are compatible if degenerate loops are confined to the most heavily degraded files.

The study covers one model, one runtime and one benchmark, and the hybrid structure, in which a few attention layers carry much of the sensitivity, may favour allocation more than a uniform transformer would. Each configuration ran the full benchmark once; `magi-1.0` with either cache ran the 126 screen tasks twice (screen and full run) with identical outputs and token counts. Two screened variants also changed the importance matrix, and the released `magi-beta-1` was not screened itself (Section 6). The footprint of `magi-beta-1` is Ollama's rounded display (3.07 ± 0.05 GiB; its tensor data suggest about 3.05 GiB). The index weights suites equally, and ToolCall is at its ceiling for every model. Random allocations were compared only on divergence, with three per budget, and on the screen, with one; no other per-tensor recipe, such as Unsloth's dynamic files or Addario's target-bpw files at 1.8–2.0 GiB, was evaluated, and the uniform-body controls are not size-matched at M and L. J was optimized and reported on the same two sets; E-code values are not available for the allocation mixtures, and the uncertainty of J is known only from the binomial standard errors in the logs, which understate it because neighbouring tokens are correlated. The behavioural screen was selected on two particular files, which the matched random control addresses but cannot remove; its tasks overlap the full benchmark, and its gate thresholds were set from `magi-1.0`'s own screen result. The loop detector has not been validated against hand-labelled traces. Throughput was recorded during benchmark runs rather than in controlled tests.

8 Conclusion

For Qwen3.5-4B on a 12 GB consumer GPU, and within weight sizes of about 1.4–2.0 GiB, the placement of bits across tensors determines much of how much capability survives quantization. Allocating standard GGUF types by measured sensitivity gives a training-free model that is smaller and more accurate than a published 2-bit file, that diverges less from BF16 than all three random allocations at each budget tested, and that behaves better on the screen than a random allocation with more bytes. Against a uniform assignment of types it ties at the smallest budget, with fewer bytes, and its margins at the larger budgets are not size-matched. Served with an 8-bit KV cache, it delivers three times the capability per gibibyte of BF16 (2.7 times with the default cache). Relative to the 2-bit file, its advantage comes almost entirely from fewer degenerate loops: it has 238 fewer looping failures but 126 more non-looping wrong answers. The same measurements indicate where the approach ends: in the configurations tested, compressing the least-sensitive tensors further came with more loops, and a longer thinking budget did not significantly improve solves. `magi-1.0` and `magi-beta-1` are available at `https://huggingface.co/GEHIRNS`.

A Candidate search

Table 9 lists every candidate of the global search of Section 6. Only candidates that passed a divergence guard, fixed before any was measured, could be screened. The search tried g, f, e and d, most aggressive first; only d passed the guard, and when it failed the gate, the pre-specified fallback, the full run of `magi-1.0` with the 8-bit cache, took its place. This order was set by hand before any candidate was measured. It replaced the rule specified first, which would have picked the candidate with the highest predicted density; under that rule no candidate would have qualified, because every candidate's predicted density was below the reference's 0.7998. Candidates a–c passed the guard but were not in the search order; x and xx were measured alongside.

Table 9: Candidates of the global search. Changes are relative to greedy M (`magi-1.0`'s types); file sizes and J are for builds with Unsloth's importance matrix (reference $J = 10.23$). Guard: $J \leq 16.36$, 1.6 times the reference.

	Change	File (GiB)	J	Guard	Screen
a	FFN of layers 28–29 at IQ1_M	1.776	10.78	pass	not run
b	a; FFN of layers 24–27, 30 at IQ2_XXS; attention k, v, output at Q6_K	1.765	11.29	pass	not run
c	b; FFN gates outside layers 8–15 at IQ2_XXS	1.718	12.31	pass	not run
d	c; FFN of layers 0, 2, 27 at IQ1_M	1.699	13.25	pass	failed[a]
e	FFN at IQ2_XXS, but layers 8–15, 31 at IQ3_XXS and 0, 2, 27–29 at IQ1_M; attention k, v, output at Q6_K	1.625	16.49	fail	—
f	e; DeltaNet output gate at IQ2_XXS, its input projections outside layers 8–23 at IQ3_XXS	1.554	17.75	fail	—
g	f; embedding at Q3_K	1.475	18.53	fail	—
x	g; FFN of layers 12, 14, 15, 24 at IQ1_M	1.440	20.30	fail	—
xx	IQ1_S throughout, but embedding Q2_K and α, β Q8_0	0.861	61.50	fail	—

[a] Built with the own-trace matrix: 45 solved, 22 loops, 29 controls; 0 of 48 chat replies flagged.

B Other interventions

Offline rotation. A Hadamard rotation folded into the FFN weights ($G' = GR^\top$, $U' = UR^\top$, $D' = RD$) preserves the function only if the FFN input is rotated as well. Because D' also rotates the FFN output, the whole residual stream would have to be rotated, which here requires untying the shared embedding and output head (0.64 B parameters) or an online transform that the runtime does not provide. Without either, the rotated model disagreed with BF16 on 99.97% of tokens, which shows only that the transform did not preserve the function; we draw no conclusion about rotation methods [2, 23].

C Reproducibility

The model files are public at `https://huggingface.co/GEHIRNS/magi-1.0` and `https://huggingface.co/GEHIRNS/magi-beta-1`; the released files have the SHA-256 prefixes `26b4f5b3` and `7b70afd2`. The files we benchmarked differ from them only in the two importance-matrix metadata strings, which held local paths; all 426 tensors are identical. The repository `https://github.com/GEHIRNS/magi` provides two folders:

- `recipe/`: the per-class type overrides of both models, the importance matrix used for both releases, a table of every tensor with its type and the SHA-256 of its data, a verification script and the build instructions (`BUILD.md`). The released files were built from the repository root with these files and the llama.cpp build named in Section 3, one `llama-quantize` call per model, so a rebuild done the same way reproduces them byte for byte. A rebuild with other paths changes only two metadata strings; all 426 tensors stay identical.
- `data/`: the numbers behind every table and figure, including per-task pass/fail, loop and budget flags for the six benchmarked configurations, per-task outcomes and paired tests for the behavioural screen, the candidates of Table 9, and the failure counts of Figure 8a. These files contain task identifiers only, which, with the selection rules given in `data/README.md`, identify the public benchmark items used.

Not yet released are the evaluation harness with its prompt templates and answer-extraction rules, the in-house ToolCall suite, the model outputs and thinking traces, the texts of the divergence sets and of the importance-matrix calibration traces (which contain problem statements), and the allocation and screening scripts.

References

[1] E. Addario. *eaddario/Qwen3.5-4B-GGUF*: target-bpw quantizations with KL-divergence tables. Hugging Face model card, 2026. `https://huggingface.co/eaddario/Qwen3.5-4B-GGUF`

[2] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, J. Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. *NeurIPS*, 2024.

[3] J. Austin, A. Odena, M. Nye, et al. Program synthesis with large language models. arXiv:2108.07732, 2021.

[4] M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.

[5] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, O. Tafjord. Think you have solved question answering? Try ARC, the AI2 reasoning challenge. arXiv:1803.05457, 2018.

[6] K. Cobbe, V. Kosaraju, M. Bavarian, et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

[7] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, K. Keutzer. HAWQ: Hessian aware quantization of neural networks with mixed-precision. *ICCV*, 2019.

[8] Z. Dong, Z. Yao, D. Arfeen, A. Gholami, M. W. Mahoney, K. Keutzer. HAWQ-V2: Hessian aware trace-weighted quantization of neural networks. *NeurIPS*, 2020.

[9] B. Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[10] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *ICLR*, 2023.

[11] G. Gerganov and contributors. llama.cpp (`tools/quantize`). Software, 2026. `https://github.com/ggml-org/llama.cpp`

[12] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, J. Steinhardt. Measuring massive multitask language understanding. *ICLR*, 2021.

[13] A. Holtzman, J. Buys, L. Du, M. Forbes, Y. Choi. The curious case of neural text degeneration. *ICLR*, 2020.

[14] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *NeurIPS*, 2024.

[15] W. Huang, H. Qin, et al. SliM-LLM: Salience-driven mixed-precision quantization for large language models. *ICML*, PMLR 267:25672–25692, 2025.

[16] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, I. Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *ICLR*, 2025.

[17] S. Kozyrev, D. Maiboroda. Why Gated DeltaNet survives 4-bit quantization: NVFP4 W4A4 for the recurrent half of a hybrid 27B LLM. arXiv:2609.04098, 2026.

[18] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *MLSys*, 2024.

[19] S. Lin, J. Hilton, O. Evans. TruthfulQA: Measuring how models mimic human falsehoods. *ACL*, 2022.

[20] J. Liu, C. S. Xia, Y. Wang, L. Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. *NeurIPS*, 2023.

[21] R. Liu, Y. Sun, M. Zhang, H. Bai, X. Yu, T. Yu, C. Yuan, L. Hou. Quantization hurts reasoning? An empirical study on quantized reasoning models. *COLM*, 2025.

[22] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, X. Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *ICML*, 2024.

[23] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, T. Blankevoort. SpinQuant: LLM quantization with learned rotations. *ICLR*, 2025.

[24] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[25] T. Mihaylov, P. Clark, T. Khot, A. Sabharwal. Can a suit of armor conduct electricity? A new dataset for open book question answering. *EMNLP*, 2018.

[26] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, T. Hashimoto. s1: Simple test-time scaling. In *Proceedings of EMNLP*, pages 20275–20321, 2025.

[27] NVIDIA. GeForce RTX 5070 family: specifications. `https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5070-family/`, accessed 2026-10-08.

[28] Ollama. Software, 2026. `https://github.com/ollama/ollama`

[29] PrismML-Eng. llama.cpp fork, `version: 0.2.0-dev` (build 10772, commit `e8fd209c9`). Software, 2026. `https://github.com/PrismML-Eng/llama.cpp`

[30] Qwen Team. Qwen3.5-4B model card and configuration. Hugging Face, 2026. `https://huggingface.co/Qwen/Qwen3.5-4B`

[31] K. Sakaguchi, R. Le Bras, C. Bhagavatula, Y. Choi. WinoGrande: An adversarial Winograd schema challenge at scale. *AAAI*, 2020.

[32] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, C. De Sa. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. *ICML*, 2024.

[33] Unsloth. Unsloth Dynamic 3.0 GGUFs; Qwen3.5 GGUF benchmarks. Documentation, 2025–2026. `https://unsloth.ai/docs/basics/dynamic-3.0-ggufs`

[34] J. Welbl, N. F. Liu, M. Gardner. Crowdsourcing multiple choice science questions. *Workshop on Noisy User-generated Text*, 2017.

[35] C. Xiao, J. Cai, W. Zhao, B. Lin, G. Zeng, J. Zhou, Z. Zheng, X. Han, Z. Liu, M. Sun. Densing law of LLMs. *Nature Machine Intelligence*, 7(11):1823–1833, 2025.

[36] A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025.

[37] S. Yang, J. Kautz, A. Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. *ICLR*, 2025.

[38] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, Y. Choi. HellaSwag: Can a machine really finish your sentence? *ACL*, 2019.

[39] X. Zheng et al. An empirical study of Qwen3 quantization. arXiv:2505.02214, 2025.